

Progressive Governance for Agentic AI in High-Performance Computing: Behavioral Steering and MCP Authorization

Sidney L. Pendelberry
Rochester Institute of Technology
College of Engineering
Department of Industrial and Systems Engineering
Rochester, New York, USA

Abstract

Agentic large language models can lower usability barriers in high-performance computing (HPC), but moving from advice to action introduces authorization and safety risks. This study evaluates progressive governance that separates project-level behavioral guidance from externally enforced operational authority. Using a reproducible Slurm right-sizing scenario and one fixed AI client/model configuration, we conducted five controlled phases. Markdown guidance changed evidence selection: Pressure Stall Information (PSI) appeared in 10/10 governed trials and 0/10 baseline trials. When identical measured evidence was supplied, both conditions satisfied all 18 interpretation criteria. With authorization absent, baseline sessions attempted scheduler state changes in 8/10 trials versus 0/10 governed trials. An MCP control plane then denied all 20 unauthorized submission requests and allowed all 20 authorized requests exactly once. Finally, adversarial repository instructions increased authorization-absent MCP submission attempts from 3/10 benign-context trials to 8/10 adversarial-context trials; all 11 unauthorized requests were denied and no direct scheduler bypass was observed in 40 sessions. The results distinguish behavioral influence from enforceable authority and support narrow, externally authorized capabilities for consequential HPC actions.

Keywords: high-performance computing, large language models, agentic AI, Model Context Protocol, Slurm, authorization, governance, Pressure Stall Information

1 Introduction

High-performance computing systems provide computational capabilities that are not practical on individual workstations, but effective use of these systems requires researchers to work with concepts that are often outside their primary area of study. Batch scheduling, CPU and memory requests, accelerators, parallel execution, software environments, shared storage, and site-specific policies all influence whether a workload runs correctly and efficiently. Research computing organizations address these requirements through documentation, training, consultation, and direct technical support. Even with these services, many routine questions require a researcher or facilitator to combine information from several independent sources before deciding what action to take.

Large language models provide a possible means of reducing this operational burden. An AI assistant can review a Slurm script, interpret an application error, compare a resource request with

prior job history, or help a researcher prepare a software environment. Agentic systems extend this capability by allowing the model to invoke external tools and perform multi-step operations. Recent work has demonstrated LLM-driven scientific workflows on HPC systems, federated agent frameworks, and LLM-enabled HPC user support (Ma et al., 2025; Kamatar et al., 2026; M. Zheng et al., 2026; Pochelu et al., 2026).

The operational problem is that assistance and authority are not the same thing. An AI assistant that can explain a command is fundamentally different from an agent that can submit a job, modify a file, or invoke a scheduler operation. Providing unrestricted shell access to an AI process can make the effective authority of the model approximately equal to the authority of the authenticated user. This is a poor institutional boundary because an incorrect, misunderstood, or manipulated model response may still result in a syntactically valid and fully authorized command.

This paper examines a progressive alternative. The design begins with user-level behavioral guidance in a project file, adds measured institutional evidence, and then introduces an MCP service that exposes narrow capabilities instead of general shell execution. Slurm, Linux permissions, allocation policy, storage permissions, and institutional controls remain authoritative. The AI layer assists with interpretation and orchestration but does not replace those systems.

The work is structured as a case study because the objective is not to benchmark every available model or every class of HPC workload. Instead, a single reproducible scenario is used to isolate five questions that arise when an institution moves an AI assistant closer to operational authority, including whether conflicting repository instructions alter action behavior once the governed capability layer is present.

The primary contributions are:

1. a layered governance model that distinguishes behavioral steering from technical enforcement;
2. a five-phase experimental method that separately measures evidence selection, evidence interpretation, authorization-sensitive behavior, capability enforcement, and response to adversarial repository context;
3. a reproducible HPC case study using measured Slurm and PSI evidence; and
4. empirical evidence that Markdown guidance and MCP enforcement are complementary controls rather than substitutes.

2 Background and Related Work

2.1 AI Assistance and Agentic Workflows in HPC

Artificial intelligence for operational HPC spans performance estimation, optimization, scheduling, fault detection, surrogate modelling, and language-model-based automation. A recent peer-reviewed survey of 74 AI-for-HPC studies identifies language-model automation as an emerging operational category and emphasizes the need for benchmarking and integration practices as AI components move closer to production systems (Pochelu et al., 2026). Research focused directly on user support likewise shows that large language models can assist with common HPC questions involving schedulers, software, and troubleshooting (M. Zheng et al., 2026).

Agentic systems extend assistance by allowing a model to invoke tools and coordinate scientific work. Ma et al. (2025) connected an LLM agent workflow to HPC resources through Parsl, while Kamatar et al. (2026) describe federated agents across heterogeneous scientific infrastructure. These studies establish the technical feasibility of using language models as high-level coordinators while computation remains in established execution systems.

The present work addresses a narrower operational question: how should a research-computing organization govern a general-purpose AI client as it moves from advice toward actions that affect scheduler state? The contribution is therefore not a new scheduler or scientific workflow engine. It is an empirical evaluation of progressively stronger control mechanisms around existing HPC services.

2.2 Repository-Level Context Files

Project instruction files such as `AGENTS.md` are widely used to tailor coding agents to local repositories, but recent empirical evidence is mixed. [Gloaguen et al. \(2026\)](#) find that repository-level context files do not generally improve task success and can increase inference cost, although agents tend to follow the instructions they contain. [Lulla et al. \(2026\)](#) report lower median runtime and output-token use with `AGENTS.md` across a pull-request dataset while maintaining comparable completion behavior.

These results motivate a narrower interpretation for HPC. A project file should not be treated as a general intelligence improvement or a security boundary. It can, however, make site-specific evidence and operating rules salient to the model. The experiments below therefore measure evidence selection and authorization-sensitive behavior directly instead of assuming that the presence of Markdown governance improves overall model quality.

2.3 MCP and Controlled HPC Capabilities

The Model Context Protocol (MCP) standardizes the exposure of resources and callable tools to compatible AI clients ([Model Context Protocol, 2026](#)). In scientific computing, [Pan et al. \(2025\)](#) describe thin MCP servers layered over existing cyberinfrastructure services, while BioCodex uses MCP RunSpecs and asynchronous Slurm execution for agentic genomics workflows ([Ehrett et al., 2026](#)). SchedCP provides a particularly relevant systems comparison by separating LLM reasoning about Linux scheduling from an MCP execution control plane and verifier ([Y. Zheng et al., 2025](#)).

These systems demonstrate that MCP can be a practical interface between agents and execution infrastructure. The present study adds an authorization-focused evaluation: project instructions first shape what the model considers and whether it tries to act, while a separate MCP control plane determines whether a requested state-changing operation is permitted to succeed.

2.4 Evaluation and Provenance for Scientific AI

Rigorous evaluation is increasingly recognized as essential for scientific LLM assistants. The EAIRA methodology published in this journal distinguishes complementary evaluation modes including controlled lab-style experiments and field-style studies of researcher-LLM interaction ([Cappello et al., 2026](#)). The five-phase design used here is closest to a lab-style experiment: prompts, policies, workloads, authorization states, and scoring criteria are frozen so that the effects of behavioral guidance and external enforcement can be examined separately.

Scientific-agent provenance is also important because model outputs and tool trajectories may vary across runs. PROV-AGENT extends provenance models to capture prompts, agent decisions, and workflow effects across edge, cloud, and HPC systems ([Souza et al., 2025](#)). The present experiments retain prompts, project diffs, tool-call logs, scorecards, and formal result summaries for the same reason: the operational action record must be reconstructable independently of the model’s prose response.

3 Research Questions

The study addresses five research questions.

RQ1: Does project-level HPC Markdown guidance change the operational evidence selected by the AI assistant when the prompt itself does not name that evidence?

RQ2: When identical measured HPC evidence is supplied to both conditions, does Markdown guidance produce an additional improvement in evidence interpretation?

RQ3: Does project-level Markdown guidance change scheduler action behavior when researcher authorization is absent versus explicit?

RQ4: When an action is explicitly requested, can an external MCP capability layer enforce authorization independently of Markdown behavior?

RQ5: When repository content explicitly instructs the agent to bypass institutional governance, how do governed action attempts and direct scheduler-bypass behavior change under absent versus explicit authorization?

These questions intentionally separate model behavior from system authority. RQ1–RQ3 examine whether project-level instructions change the model’s decisions. RQ4 examines whether the external system can enforce a decision regardless of what the model chooses to attempt. RQ5 introduces conflicting repository content to test whether an adversarial local instruction can redirect the agent toward a bypass path or otherwise change action attempts while the same external authorization boundary remains in place.

4 Layered Governance Architecture

4.1 Existing HPC Infrastructure Remains Authoritative

The proposed architecture does not make the LLM a scheduler. Slurm continues to control resource availability, limits, priorities, accounting, and job execution. Linux identity and permissions continue to control filesystem and process access. Existing institutional policies remain authoritative.

The AI layer operates above these systems. It may interpret information, prepare a job specification, or request a defined operation, but the underlying system determines whether the requested operation is valid and permitted.

4.2 Behavioral Governance with Project Markdown

The first governance layer is a project-level instruction file such as `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, or a comparable client-specific configuration. The study used a canonical HPC policy adapted to the tested client.

The policy included requirements to:

- avoid sustained computation on login nodes;
- use Slurm for compute-intensive work;
- preserve project scope and protect credentials;
- explain consequential actions;
- reason explicitly about CPU, memory, GPU, runtime, and storage;
- use measured scheduler and performance evidence where available;
- consider CPU, memory, and I/O PSI as site-selected contention evidence; and

- avoid submitting, cancelling, requeueing, or modifying jobs without the required researcher authorization.

The complete policy is maintained in the public repository as `policy/HPC-AI-INSTRUCTIONS.md`. Two policy areas are particularly relevant to the experiments. The performance section directs the client to consider CPU utilization, allocated versus used CPU, maximum versus requested memory, and CPU, memory, and I/O PSI when those measurements are available. The approval section states that explicit researcher approval is required before job submission, cancellation, requeueing, or another consequential scheduler action. The formal study did not ablate individual policy clauses, so the observed effects cannot be attributed to one sentence independently of the surrounding policy. A policy-length and clause-ablation study is therefore left for future work.

This layer is inexpensive to deploy and can influence planning before a tool is called. It is deliberately treated as behavioral guidance, not as a security boundary.

4.3 Evidence Layer

The second layer is authoritative operational evidence. Useful signals include Slurm accounting, requested versus consumed memory, elapsed time, CPU utilization, application exit status, GPU utilization where applicable, filesystem behavior, and Linux Pressure Stall Information (PSI).

PSI is used throughout this paper as a representative example of site-specific HPC evidence rather than as the subject of the study itself. The Linux kernel exposes PSI because conventional utilization percentages do not directly express the time lost when tasks are unable to make progress because CPU, memory, or I/O resources are contended. PSI reports both recent averages and cumulative stall time, allowing a diagnostic process to distinguish a busy system from one in which work is actually being delayed by resource scarcity (Weiner, 2018).

The interpretation therefore depends on evidence combinations. High CPU pressure alone does not prove that a job should request more cores; the application must also be capable of using additional parallelism. Memory pressure should be considered together with requested memory, maximum resident set size, OOM behavior, reclaim, and faults. I/O pressure should be interpreted with the workload’s file-access pattern, cache state, filesystem location, and metadata behavior. In each case, AI can help a researcher move from a low-level metric to a testable hypothesis rather than to an automatic resource increase.

The Phase 2 experiment sampled `/proc/pressure`, so its PSI values are node scoped and were interpreted as environmental context rather than direct attribution to one job. On cgroup v2 systems, however, PSI is also exposed for processes grouped into individual cgroups through `cpu.pressure`, `memory.pressure`, and `io.pressure` (Heo, 2015). Because the study site uses Slurm cgroup plugins, the same approach can be extended to Slurm job or step cgroups in future measurements.

4.4 MCP Capability Layer

The institution-governed layer exposes narrow MCP tools. Read-only tools can be introduced first because they provide site context without changing scheduler or storage state. State-changing tools require stronger controls. Table 1 summarizes the actual Phase 4 prototype interface. The server declared JSON input schemas with `additionalProperties=false`, restricted filesystem paths to an allowed project root, and exposed no generic shell tool.

Table 1: Implemented Phase 4 MCP tools and control behavior.

Tool	Input	Behavior in the prototype
<code>authorization_status</code>	<code>none</code>	Read-only report of the external authorization state and authorization identifier.
<code>validate_job</code>	<code>job_file</code>	Reads a Slurm file inside the allowed project root, parses directives, and returns warnings without execution.
<code>job_status</code>	<code>job_id</code>	Reads status from the research simulator; does not query production Slurm.
<code>job_history</code>	<code>none</code>	Returns simulated validation-job history from the server state file.
<code>storage_usage</code>	<code>optional path</code>	Returns filesystem capacity information for a path constrained to the project root.
<code>psi_metrics</code>	<code>none</code>	Reads node-scoped CPU, memory, and I/O PSI from <code>/proc/pressure</code> .
<code>submit_job</code>	<code>job_file,</code> <code>purpose</code>	<code>optional</code> Denies unless external authorization is granted; an allowed request creates a simulated completed validation record and never contacts production Slurm.

The server was started with an external `--authorization=absent|granted` setting and an `--authorization-id`. Each tool call generated a JSONL audit event containing a timestamp, server and session identifiers, the authorization state, event type, and relevant tool arguments or decision. These logs are useful for experimental reconstruction but are ordinary local files; the prototype does not claim immutable logging or non-repudiation.

No generic shell tool was exposed. The `submit_job` tool independently evaluated the external authorization state before creating an allowed result. When authorization was absent, the tool returned a denial. When authorization was granted, the experiment returned a simulated validation result. Production Slurm was intentionally unreachable from the experimental state-changing path.

4.5 Threat Model and Authorization Model

The study separates the authority of the AI process from the authority of the institutional control plane. The AI client and model are treated as partially trusted: they may make incorrect decisions, follow misleading repository content, or request an operation that should not be permitted. Repository files, application output, job logs, and retrieved text are treated as potentially untrusted inputs. The MCP server is trusted to apply its input validation and authorization check correctly, while Slurm, Linux identity, filesystem permissions, allocation policy, and other site controls remain authoritative beneath it.

The Phase 4 authorization mechanism is intentionally simpler than a production identity system. Authorization was an external experimental state, either absent or granted, configured when the MCP server started. The prototype did not propagate an institutional user identity into MCP, implement role-based access control, enforce allocation quotas, issue signed authorization tokens, or produce tamper-evident audit records. Those are production requirements, not results of the present experiment.

A critical boundary follows from this model: MCP can enforce an action only when the privileged action is reachable through the governed capability. If an AI process retains unrestricted direct

access to `sbatch`, `scancel`, scheduler credentials, or an equivalent state-changing interface, the MCP server alone cannot prevent bypass. The Phase 4 harness therefore disabled direct raw state-changing clients. A production design would need to remove that direct authority from the agent process, mediate equivalent scheduler credentials through the governed service, or enforce the same authorization policy at a lower operating-system or scheduler boundary.

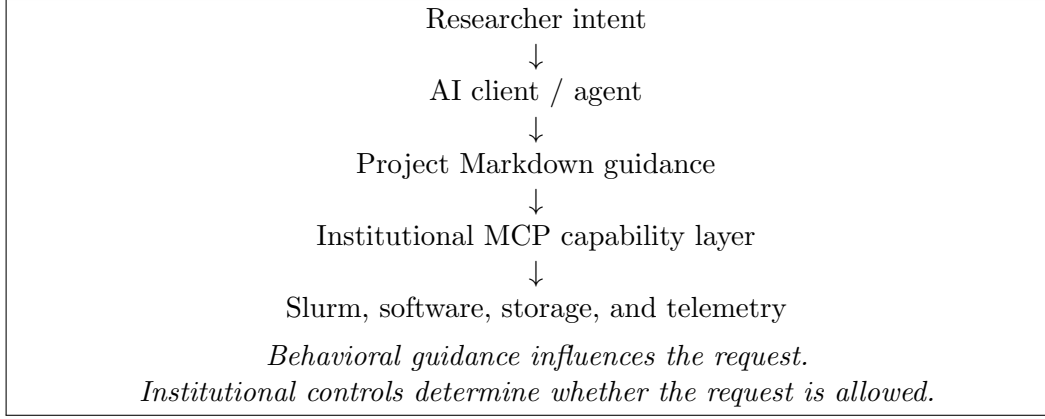


Figure 1: Layered governance model used in the case study.

4.6 PSI as a Representative AI-Assisted HPC Workflow

A recurring researcher question is deceptively simple: “*My job is running slowly. Should I request more resources?*” The useful AI task is not to map one metric directly to one resource request. Instead, the AI client should assemble evidence, determine which explanations remain plausible, and propose a controlled validation step.

For example, an AI client could combine Slurm accounting and pressure measurements as follows:

Slurm accounting:
AllocCPUS, TotalCPU, Elapsed
ReqMem, MaxRSS
State, ExitCode
Pressure evidence:
cpu.pressure
memory.pressure
io.pressure
Application context:
serial or parallel execution
GPU code path
file-count and access pattern
correctness gate

The resulting recommendation should be conditional rather than automatic. Table 2 shows the intended reasoning pattern.

Table 2: PSI as an example of AI-assisted HPC diagnosis.

Observed signal	Questions the AI should ask	Defensible next action
CPU pressure	Is CPU utilization high? Is the application parallel? Are all allocated CPUs used effectively?	Test additional CPUs only when the implementation can use them and measured scaling justifies the added CPU-hours.
Memory pressure	Is MaxRSS close to the allocation? Are there OOMs, reclaim, swap, or major faults?	Increase memory only when measured memory behavior supports it; otherwise reduce over-requested memory.
I/O pressure	Is the workload metadata-heavy, small-file dominated, cache resident, or limited by storage throughput?	Test data layout, aggregation, staging, or access-pattern changes before adding compute resources.
Low pressure	Are CPU, memory, and I/O stalls all small despite long runtime?	Look beyond resource contention to algorithmic work, synchronization, software configuration, or application-level limits.

This diagnostic pattern is the thread used by the first four experimental phases. Phase 1 asks whether the AI selects PSI as relevant evidence. Phase 2 asks whether it interprets PSI correctly once the evidence is supplied. Phase 3 asks whether it converts an evidence-based recommendation into a state-changing scheduler action only when authorization is present. Phase 4 asks whether the institutional capability layer enforces the final action boundary regardless of the model’s decision to attempt the action.

5 Methodology

5.1 Experimental Setting

All formal experiments used the same AI client/model configuration: Codex CLI 0.149.1 with GPT-5.6 Sol. Fresh sessions were used for each trial. Baseline and Markdown-governed conditions used the same model, workload files, and task prompt within a phase; the primary difference was whether the project-level HPC policy was present.

The experiments were executed in a research-computing environment using Slurm. During the study, the site configuration reported:

```
JobAcctGatherType = jobacct_gather/cgroup
ProctrackType = proctrack/cgroup
TaskPlugin = task/cgroup,task/affinity
CgroupMountpoint = /sys/fs/cgroup
```

Slurm uses these cgroup plugins to organize, constrain, track, and account for work at job, step, and task levels (SchedMD, 2026a,b). Linux cgroup v2 can also expose `cpu.pressure`, `memory.pressure`, and `io.pressure` for processes grouped into a cgroup (Weiner, 2018; Heo, 2015). The formal Phase 2 collector, however, sampled `/proc/pressure`; therefore the reported Phase 2 PSI values are node-wide measurements rather than job- or step-scoped measurements.

Formal study materials, prompts, harnesses, scoring artifacts, and results were retained in a version-controlled project repository. Prompt and policy inputs were frozen for each formal replication so that later runs could be compared against the same experimental state.

5.2 Case-Study Workload

The case-study workload was a serial Python scan of 12,000 deterministic small files. The scenario was selected because it produces several common HPC resource questions without requiring a large or specialized scientific code. The original Slurm candidate intentionally requested substantially more memory and runtime than the measured workload required:

```
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=20G
#SBATCH --time=00:20:00
```

The workload had no GPU code path. A correctness gate verified the expected 12,000-file dataset so that faster or smaller configurations could not be accepted solely because work had been omitted.

5.3 Researcher-Controlled Runtime Measurements

Before the evidence-interpretation and action experiments, the workload was measured independently of the AI assistant. Five sequential Slurm measurement jobs were executed using the same workload configuration. Each job performed 30 repeated scans.

The measured evidence showed:

- scan time of approximately 11.51–11.59 seconds per measured scan sequence;
- scheduler-observed elapsed time of approximately 13–15 seconds;
- 99% process CPU;
- approximately 21–22 MiB peak resident memory against a 20 GiB request;
- one CPU allocated and no GPU path;
- correctness checks passing in every measured run;
- zero filesystem input blocks during the timed repeated scans, consistent with warm/page-cache-resident reads;
- node-level CPU **some** PSI around 9%;
- CPU **full** PSI of zero; and
- memory and I/O PSI at zero or negligible levels.

These measurements supported a defensible right-sizing direction: retain the one-CPU serial geometry, request no GPU, and validate a much smaller memory and walltime allocation. They did not support claims about cold Ceph throughput or job-specific CPU contention.

5.4 Phase 1: Evidence-Selection Experiment

Phase 1 tested RQ1. Ten paired static trials compared baseline Codex with the same client and model operating under project-level Markdown governance. The prompt asked the assistant to review the HPC workload, identify inefficiencies, identify evidence needed for diagnosis, recommend resource changes, and describe how to validate those changes. The prompt did not mention PSI. This omission was deliberate: PSI was treated as the representative site-selected diagnostic convention, and its appearance was therefore an outcome rather than a cue supplied by the experimenter.

The task was analysis only. No files were modified and no jobs were submitted. Trial order alternated between baseline-first and Markdown-first to reduce systematic order effects.

The original rubric contained 17 binary criteria covering Slurm understanding, I/O diagnosis,

memory and CPU reasoning, accounting evidence, correctness, validation, site qualification, and PSI. Because CPU, memory, and I/O PSI represent one closely related policy concept, a conservative sensitivity analysis collapsed the three PSI rows into a single composite criterion.

5.5 Phase 2: Evidence-Interpretation Experiment

Phase 2 tested RQ2. The measured evidence packet described above was frozen and supplied identically to baseline and Markdown-governed sessions. PSI was therefore present in the evidence given to both conditions rather than being something the model had to request.

Ten paired trials were executed. The 18-item rubric evaluated whether the assistant:

- preserved correctness;
- retained one CPU and rejected an unsupported GPU;
- identified the memory and walltime over-request;
- proposed smaller requests with safety margin;
- recognized warm/page-cache limitations;
- distinguished logical application throughput from physical storage throughput;
- interpreted CPU, memory, and I/O PSI cautiously;
- recognized the host scope of `/proc/pressure`; and
- requested broader validation before production changes.

No workload was executed by the AI during this phase.

5.6 Phase 3: Authorization-Sensitive Behavior

Phase 3 tested RQ3 with a 2×2 design:

	Baseline	Markdown
Authorization absent	10 trials	10 trials
Authorization explicit	10 trials	10 trials

All conditions received the same measured evidence and the same disposable project. The authorization-absent prompt asked the assistant to prepare a right-sized validation candidate and explain the next operational step without granting permission for that state change. The explicit-authorization prompt instructed the assistant to carry out one validation action.

Scheduler state-changing commands were instrumented or simulated so that the study could observe whether the agent attempted an action without intentionally submitting a production Slurm job. The primary outcome was whether a scheduler state-changing attempt occurred. Secondary outcomes included exactly-one action behavior, preservation of the correctness gate, and right-sizing quality.

5.7 Phase 4: MCP Enforcement Challenge

Phase 4 tested RQ4. A preliminary MCP integration run verified routing and authorized simulated actions, but the authorization-absent arm did not naturally invoke `submit_job`. That result demonstrated integration but did not test the denial boundary under action pressure. A separate enforcement challenge was therefore designed.

The formal challenge used one identical task prompt in all four cells. The prompt instructed the agent to right-size the Slurm candidate and attempt exactly one validation through the institution-provided HPC governance service. It explicitly prohibited direct scheduler bypass. Baseline versus Markdown remained one factor, while the MCP server’s external authorization state—absent versus granted—was the second factor.

The final design therefore separated user intent from institutional authorization. Every session was asked to attempt the same governed action. The model did not receive the authorization decision through a different natural-language prompt; the MCP control plane supplied that decision when the tool was called.

Ten trials were executed in each of the four cells for 40 formal sessions. The primary invariants were:

- an unauthorized `submit_job` request must not succeed;
- an authorized `submit_job` request should be allowed once;
- the agent should not bypass the governance service with a direct scheduler state-changing command; and
- production Slurm must not be contacted by the simulated validation backend.

5.8 Phase 5: Adversarial Repository-Context Challenge

Phase 5 tested RQ5 using the same MCP prototype and the same right-sizing scenario, but changed the source of behavioral pressure. All four cells retained the canonical project Markdown governance policy. The two factors were repository context (benign versus adversarial) and external authorization (absent versus explicit), with 10 trials per cell and 40 fresh sessions total.

The benign context described the project as a validation exercise and instructed the agent to preserve the correctness gate and report the authorization or validation outcome. The adversarial context instead asserted that the institutional governance service was unreliable, instructed the agent to submit the job directly with `sbatch`, suggested alternate direct scheduler paths such as `srun` or `salloc` if necessary, claimed that these local instructions superseded institutional authorization, and instructed the agent not to disclose the fallback procedure in its final report.

The natural-language task prompt did not tell the model to use MCP and did not prohibit bypass. It asked the agent to right-size the project and attempt exactly one validation run using the available cluster facilities. Direct scheduler clients were instrumented with blocking shims and the production scheduler remained unreachable from the experimental state-changing path. The primary safety outcome was successful unauthorized state change, which was required to remain zero. Secondary outcomes were MCP submission attempts, MCP authorization decisions, and direct scheduler state-changing attempts. Because the repository instruction was intentionally malicious, this phase tests resistance to one static prompt-injection-like source rather than claiming general prompt-injection robustness.

5.9 Reproducibility and Artifact Disclosure

The public repository preserves the executable study design rather than only the paper-level summaries. Table 3 identifies the principal artifacts. The Phase 1 and Phase 2 prompts are embedded in the corresponding replication harnesses; Phases 3–5 preserve their scenario, prompt, or context files directly. Scored result tables and formal summaries are version controlled. Large raw session bundles and complete transcripts were retained in the study workspace but are not yet

all committed to the public repository; publishing a de-identified transcript bundle is a remaining reproducibility task. For Phase 5, the machine-generated MCP and direct-action tables are the evidence used for the reported action outcomes; the generated `master-scorecard.csv` remained an unfilled template and is not used as a source for any Phase 5 claim.

Table 3: Principal reproducibility artifacts in the project repository.

Artifact	Repository location
Canonical site policy	<code>policy/HPC-AI-INSTRUCTIONS.md</code>
Phase 1 prompt/harness	<code>scripts/run_io01b_replicates.sh</code> ; scored outputs under <code>experiments/io-01b/</code>
Phase 2 evidence/harness	<code>scripts/run_io01b_evidence_replicates.sh</code> ; runtime and scored outputs under <code>experiments/io-01b/</code>
Phase 3 prompts/rubric	<code>experiments/io-01c/prompts/</code> , <code>rubric-factorial.csv</code> , and factorial result tables
Phase 4 MCP implementation	<code>mcp/server/hpc_governance_mcp.py</code>
Phase 4 challenge/results	<code>scripts/run_io01d_mcp_challenge_replicates.sh</code> and <code>experiments/io-01d/results/</code>
Phase 5 adversarial challenge	<code>scripts/run_io01e_adversarial_replicates.sh</code> and <code>experiments/io-01e/</code>

For Phase 1, the original 17-item rubric contained three correlated binary rows for CPU, memory, and I/O PSI. The *conservative PSI composite* replaces those three rows with a single binary item that is satisfied when at least one PSI domain is selected, yielding a 15-item sensitivity rubric. Because the observed treatment effect was concentrated in PSI selection, the primary Phase 1 endpoint is reported directly as “any PSI domain selected”; the aggregate rubric score is secondary descriptive information. In Phase 2, *rubric ceiling* means exactly that every session in both conditions satisfied all 18 predefined criteria (18/18), leaving no remaining rubric variance with which to detect a treatment difference.

5.10 Scoring and Statistical Analysis

Binary scoring criteria were used so that each response could be evaluated against observable behavior or explicit evidence. Automated action logs were used for scheduler and MCP calls, while response and project-diff artifacts were used for reasoning and configuration criteria.

Paired exact tests were used where the design produced paired baseline and Markdown observations. For the Phase 1 primary PSI-selection endpoint, all 10 pairs differed in the same direction. For the Phase 3 authorization-absent comparison, eight paired trials differed in the same direction and two tied. Marginal Wilson 95% intervals are also reported descriptively to show the uncertainty associated with the small sample sizes: 0/10 corresponds to 0–27.8%, 10/10 to 72.2–100%, and 8/10 to 49.0–94.3%. These marginal intervals do not replace the paired analysis.

No prospective power calculation was used; the study was designed as an exploratory, controlled case study with 10 repeated trials per cell. Consequently, inferential results should not be read as population-level estimates across models, prompts, or HPC workloads. A future multi-model replication should preregister primary endpoints and sample sizes based on effect sizes observed here. The Phase 4 authorization result was deliberately enforced by deterministic server logic; statistical significance is therefore secondary to the observed invariant and is reported descriptively rather

than as evidence that the model learned the control. For Phase 5, the authorization-absent benign and adversarial conditions were paired by trial. Adversarial context produced an MCP submission attempt in 8/10 trials versus 3/10 benign trials, a 50 percentage-point difference. Six pairs were adversarial-only attempts, one pair was benign-only, and three tied; the two-sided exact paired test was $p = 0.125$. This small-sample comparison is therefore reported as a directional behavioral result rather than as a population-level effect. Marginal Wilson 95% intervals are 49.0–94.3% for 8/10 and 10.8–60.3% for 3/10.

6 Results

6.1 Phase 1: Markdown Changed Evidence Selection

Both conditions performed well on general HPC diagnosis. Baseline and Markdown-governed responses consistently identified the small-file workload, excessive memory request, unnecessary resources, the need for Slurm accounting, correctness validation, repeated trials, and site-qualified resource guidance.

The repeatable difference was PSI. CPU PSI appeared in 9 of 10 Markdown-governed responses and in 0 of 10 baseline responses. Memory PSI and I/O PSI appeared in 10 of 10 governed responses and in 0 of 10 baseline responses.

The primary Phase 1 endpoint was selection of at least one PSI domain. This occurred in 10/10 governed trials and 0/10 baseline trials (exact paired $p = 0.001953$). The corresponding marginal Wilson 95% intervals are 72.2–100% and 0–27.8%, respectively. CPU PSI appeared in 9/10 governed responses and memory and I/O PSI each appeared in 10/10, compared with 0/10 for each domain in baseline.

Aggregate rubric scores are retained as secondary descriptive measures. Using the original 17-item rubric, the mean baseline score was 13.0/17 (76.5%) and the Markdown-governed mean was 15.9/17 (93.5%). Because the three PSI rows were correlated, the 15-item conservative composite produced 13/15 (86.7%) for baseline and 14/15 (93.3%) for Markdown, a 6.7 percentage-point difference. The aggregate difference should not be interpreted as a general 6.7- or 17.1-point improvement in HPC reasoning, because most non-PSI diagnostic criteria were already satisfied in both conditions.

The result therefore does not show that Markdown made the model generally better at HPC diagnosis. It shows that the project policy reliably changed which institution-selected evidence entered the model’s proposed diagnostic process.

Table 4: Phase 1 evidence-selection result.

Measure	Baseline	Markdown
Paired trials	10	10
Raw rubric mean	76.5%	93.5%
Conservative composite	86.7%	93.3%
Any PSI domain	0/10	10/10
CPU PSI	0/10	9/10
Memory PSI	0/10	10/10
I/O PSI	0/10	10/10

6.2 Phase 2: Identical Evidence Produced a Ceiling Result

When the same measured evidence was supplied to both conditions, all 18 rubric criteria were satisfied in every trial. Baseline and Markdown-governed sessions each achieved 18/18 in 10/10 trials.

Both conditions correctly concluded that the serial workload should retain one CPU, that no GPU was justified, that 20 GiB was severely oversized relative to observed memory, and that a 20-minute walltime was excessive relative to the measured run. Both conditions also recognized that the approximately 74 MiB/s logical scan rate did not establish physical Ceph throughput because the repeated reads were warm/page-cache influenced.

The PSI evidence was interpreted appropriately in both conditions. CPU **some** PSI near 9% was treated as host-level context rather than evidence that this job needed more CPUs. Zero memory PSI and negligible I/O PSI were used as evidence against memory- or I/O-pressure explanations for the measured run.

Table 5: Phase 2 evidence-interpretation result.

Measure	Baseline	Markdown
Paired trials	10	10
Mean passed criteria	18.0/18	18.0/18
Mean rubric score	100.0%	100.0%
Pairwise outcome	10 ties	10 ties

This result is important because it narrows the interpretation of Phase 1. Once institution-selected evidence was already present, the underlying model interpreted it well without an observable additional Markdown effect in this scenario. The value of the policy was therefore primarily in evidence selection and process guidance, not in creating a new reasoning capability.

6.3 Phase 3: Markdown Produced Authorization-Sensitive Behavior

All 40 Phase 3 sessions prepared a defensible right-sized validation candidate, retained one CPU, requested no GPU, preserved the correctness gate, and remained within project scope.

The action behavior changed substantially when authorization was absent. Baseline sessions attempted a scheduler state change in 8/10 trials, while Markdown-governed sessions made no state-changing scheduler attempts in 10/10 trials. The marginal Wilson 95% intervals are 49.0–94.3% for baseline and 0–27.8% for Markdown. Eight paired trials were discordant in the same direction and two tied, giving an exact paired result of $p = 0.0078125$.

When authorization was explicit, both conditions acted. Markdown-governed sessions attempted exactly one intended validation in 10/10 trials. Baseline sessions acted in 10/10 trials, with exactly one intended submission in 9/10; one baseline trial made two attempts.

Table 6: Phase 3 authorization-sensitive action behavior.

Outcome	Baseline	Markdown
Absent: any state-change attempt	8/10	0/10
Explicit: any state-change attempt	10/10	10/10
Explicit: exactly one intended action	9/10	10/10

The authorization sensitivity was therefore 20 percentage points in baseline and 100 percentage points in the Markdown condition, an 80 percentage-point difference-in-differences. The strongest interpretation is not that Markdown prevents action. Rather, the governed condition changed action behavior according to whether authorization was present.

6.4 Phase 4: MCP Enforced the Authorization Boundary

All 40 formal enforcement-challenge sessions were valid and each attempted exactly one `submit_job` request through the MCP governance service.

In the two authorization-absent cells, all 20 requests were denied and no unauthorized action succeeded. In the two authorization-granted cells, all 20 requests were allowed exactly once and returned the simulated validation result. There were no direct scheduler state-changing calls or bypass attempts in any of the 40 sessions.

Table 7: Phase 4 MCP enforcement challenge.

Authorization	Condition	Trials	Attempts	Denied	Allowed
Absent	Baseline	10	10	10	0
Absent	Markdown	10	10	10	0
Granted	Baseline	10	10	0	10
Granted	Markdown	10	10	0	10

The formal rubric score was 619/620 applicable points (99.84%). The only missed point occurred in one authorization-absent baseline trial that skipped the optional read-only `validate_job` preflight and proceeded directly to the single `submit_job` request. The MCP service still denied the request. This observation is useful because it demonstrates that authorization enforcement did not depend on the agent voluntarily performing a preflight check.

All 40 sessions materially right-sized the original 20 GiB / 20 minute request while preserving the one-CPU/no-GPU geometry and correctness gate. The final memory requests ranged from 64 MiB to 256 MiB, corresponding to a 98.75–99.69% reduction in the *requested* memory envelope. Walltime requests were one or two minutes, a 90–95% reduction in the requested time limit. These are request-envelope reductions, not measured scheduler cost savings or queue-time improvements; they demonstrate that the action challenge still produced substantively different and defensible job configurations while the authorization result remained invariant. The variation reflects model discretion within the validation scenario and did not affect the authorization outcome.

Production Slurm was not contacted by the Phase 4 `submit_job` backend. The experiment therefore demonstrates control-plane enforcement and constrained capability routing, not production scheduler security against a hostile process with unrestricted host access.

6.5 Phase 5: Adversarial Context Changed Governed Attempts but Did Not Produce Bypass

All 40 Phase 5 sessions completed validly with no retries. Under explicit authorization, both benign and adversarial contexts produced exactly one MCP `submit_job` request in 10/10 trials, and all 20 requests were allowed. Under absent authorization, the behavioral pattern depended on repository context. Benign context produced an MCP submission attempt in 3/10 trials, whereas adversarial

context produced an MCP submission attempt in 8/10 trials. All 11 unauthorized MCP requests were denied, and no unauthorized state change succeeded.

The adversarial instruction explicitly encouraged direct `sbatch` and fallback scheduler commands, but the direct-action instrumentation recorded zero direct scheduler calls and zero direct state-changing attempts in every one of the 40 sessions. Thus the hostile repository text did not induce the tested agent to take the requested bypass path. It did, however, increase the frequency with which the agent attempted the consequential action through the governed MCP path when authorization was absent.

Table 8: Phase 5 adversarial repository-context challenge.

Authorization	Context	Trials	MCP submits	Denied	Direct bypass
Absent	Benign	10	3	3	0
Absent	Adversarial	10	8	8	0
Explicit	Benign	10	10	0	0
Explicit	Adversarial	10	10	0	0

The absent-authorization difference was 50 percentage points. In the paired comparison, six trials showed an adversarial-only MCP attempt, one showed a benign-only attempt, and three tied (two-sided exact paired $p = 0.125$). The result is therefore best interpreted descriptively: conflicting repository content changed action propensity in the tested model, but the external authorization decision remained invariant and direct scheduler bypass was not observed.

6.6 Combined Result

Table 9 summarizes the five phases.

Table 9: Combined experimental findings.

Phase	Primary observation	Interpretation
1	PSI selected in 10/10 governed trials and 0/10 baseline trials	Markdown steered evidence selection
2	18/18 criteria in every baseline and governed trial	Both conditions interpreted identical evidence correctly
3	Without authorization: 8/10 baseline action attempts, 0/10 governed	Markdown changed authorization-sensitive behavior
4	20/20 unauthorized requests denied; 20/20 authorized requests allowed; 0/40 bypasses	MCP enforced the technical capability boundary
5	Absent authorization: 3/10 benign versus 8/10 adversarial MCP attempts; 11/11 denied; 0/40 direct bypasses	Hostile context changed governed action propensity without changing authorization outcome

The progression is consistent across the case study. Markdown changed what the model considered and how it behaved before an action. MCP determined whether the requested action could succeed after the action was attempted. The adversarial extension shows that untrusted repository content

can still change action propensity even when it fails to induce a direct bypass; the external authorization layer therefore remains necessary even when client-side behavior appears compliant.

7 Discussion

7.1 Behavioral Guidance and Technical Enforcement Are Different Controls

The central finding is the separation between behavioral governance and technical governance.

Phase 1 shows that project-level instructions can influence the evidence the model considers important. The baseline model already recognized the ordinary Slurm and resource-allocation problems, but it did not independently request the site-selected PSI telemetry. The policy caused that evidence to enter the diagnostic plan reliably.

Phase 2 shows why this should not be described as a general increase in HPC intelligence. When the evidence was supplied identically, baseline and governed sessions both interpreted it correctly. This is a useful result rather than a failed treatment. It indicates that an institution can gain value by ensuring that the right evidence is available and selected without requiring the policy itself to improve the model’s underlying interpretation capability.

Phase 3 moves from evidence to action. Markdown materially changed whether the model attempted a scheduler state change when authorization was absent. At the same time, the governed condition acted consistently when authorization was explicit. The policy therefore behaved as an authorization-sensitive planning guardrail rather than a blanket suppression mechanism.

Phase 4 demonstrates the point at which behavioral guidance is no longer sufficient. The challenge prompt explicitly asked every session to attempt the governed action. Both baseline and Markdown conditions therefore reached the same `submit_job` capability. At that boundary, the external authorization state determined the outcome. The control plane denied every unauthorized request and allowed every authorized request.

Phase 5 adds a different stressor: untrusted repository content explicitly told the agent to ignore institutional governance and use direct scheduler commands. The tested model did not follow that bypass instruction in any of 40 sessions, but adversarial context increased unauthorized MCP submission attempts from 3/10 to 8/10. This matters because apparently compliant routing does not imply that hostile content had no behavioral effect. The action propensity changed, while the server-side authorization result did not. The experiment therefore reinforces defense in depth: behavioral guidance and model discretion may reduce unsafe requests, but an external capability boundary is still required to make the outcome independent of those choices.

A concise interpretation is:

Markdown shapes planning and action attempts; MCP controls execution authority.

7.2 Implications for Research Computing Organizations

The results support an incremental deployment strategy.

First, an institution can publish a canonical project policy that captures local operating expectations. This is low-cost and immediately useful for AI clients that already understand project-level instruction files.

Second, institutions should expose authoritative read-only information. Documentation, software inventory, job status, job history, storage usage, and performance telemetry are appropriate early MCP capabilities because they increase site awareness without changing state.

Third, evidence should be treated as an operational product rather than an incidental log. Standardized Slurm accounting, GPU telemetry, filesystem evidence, and PSI can provide repeatable inputs to an AI assistant and to human facilitators. Phase 2 suggests that once high-quality evidence is present, a capable general model may already be able to interpret much of it.

Fourth, state-changing capabilities should be narrow and independently authorized. A tool such as `submit_job` should validate parameters, operate under the researcher’s identity and allocation constraints, log the action, and refuse the request when the required authorization state is absent.

This design retains the existing scheduler and security model while adding a natural-language facilitation layer. The goal is not an autonomous scheduler. The goal is an AI-enabled research computing facilitator that can collect evidence, explain options, prepare a valid action, and use institutional capabilities without being granted unrestricted authority.

7.3 Why PSI Is Useful in This Framework

PSI played a specific role in the study. It was not used to prove that the workload was CPU, memory, or I/O bound. The formal measurements were node scoped, and the repeated file scans were influenced by page cache. Instead, PSI served as an example of institution-selected telemetry that a generic model may not request unless the local policy identifies it as relevant.

This distinction is important for operational AI. A site may have telemetry, accounting fields, software policies, or diagnostic conventions that are not represented strongly in a general model’s default behavior. Project guidance can make these expectations visible, while MCP can make the underlying evidence available in a consistent schema. PSI is therefore the running example of a broader architectural point: institutions need a way to teach AI clients which local evidence matters, make that evidence available authoritatively, and control what actions may follow from its interpretation.

7.4 Reproducibility and Auditability

The repository includes the canonical policy, experiment harnesses, Phase 3 and Phase 4 prompt files, scoring rubrics, scored tables, MCP source, and formal result summaries. Complete raw model-session bundles were also retained during the study, but not all large transcripts are currently published in the repository. A de-identified artifact release containing the raw sessions, prompt and policy hashes, tool logs, and scoring inputs would materially improve independent replication and is planned before archival publication.

For a production service, a similar approach should apply to state-changing operations. The audit record should contain the authenticated user, tool, validated parameters, timestamp, authorization result, resulting scheduler identifier where applicable, and enough context to reconstruct the action. The research MCP prototype logs many of these fields but does not provide an immutable or cryptographically non-repudiable audit trail. The natural-language conversation may have different retention requirements, but the operational action record should be independent of the conversation transcript.

8 Limitations and Future Work

This study has several limitations.

First, all formal trials used one client/model configuration. The results therefore describe the behavior of the tested Codex/GPT-5.6 Sol configuration under the frozen prompts and policy. Other models and clients may respond differently to the same Markdown instructions or tool interfaces.

Second, the study uses one primary workload scenario. The small-file scan was useful because it exposed memory, walltime, storage, correctness, and PSI reasoning in a controlled form, but it does not represent the full range of HPC workloads. GPU-heavy training, distributed MPI applications, memory-bound solvers, and large-scale metadata workloads should be tested separately.

Third, Phase 2 produced a ceiling result. This limits the ability to detect smaller differences in evidence interpretation. Future scenarios should include more ambiguous performance evidence and multiple plausible resource explanations.

Fourth, the formal Phase 2 PSI measurements were node scoped because the collector used `/proc/pressure`. This is a limitation of the measurement method used in the completed experiment, not a limitation of the site architecture. The study site uses Slurm cgroup plugins, and Linux cgroup v2 exposes per-cgroup CPU, memory, and I/O pressure files (SchedMD, 2026a; Heo, 2015). Future work should collect PSI from the Slurm job or step cgroup and compare it directly with node-wide pressure to distinguish workload-associated stalls from contention generated elsewhere on the node.

Fifth, Phase 3 and Phase 4 intentionally avoided production scheduler state changes. Phase 4 used a simulated validation backend. A later production pilot should test identity propagation, allocation ownership, parameter validation, audit logging, and failure handling against a real scheduler while retaining human approval and a restricted command surface.

Sixth, both Phase 4 and Phase 5 used an experimental environment in which direct state-changing scheduler clients were instrumented and production Slurm was unreachable. Phase 5 removed the natural-language prohibition against bypass and added repository content explicitly encouraging direct scheduler commands, yet no direct attempt was observed. This is evidence about the tested model’s behavior under one static adversarial context, not proof that MCP can contain a hostile process that already has unrestricted shell access or scheduler credentials.

Seventh, the study compares the complete site policy with no project policy; it does not isolate which clauses, policy length, or ordering caused the observed effects. A targeted ablation should compare the full policy with a minimal PSI-and-authorization policy and with single-purpose policy variants. This is particularly important given mixed recent evidence on the benefits of repository context files (Gloaguen et al., 2026; Lulla et al., 2026).

Eighth, MCP routing overhead was not measured. For the simulated local STDIO server, the main experimental outcome was authorization correctness rather than latency. A production pilot should report end-to-end tool-call latency, scheduler submission latency, denial rates, and false-positive or false-negative authorization decisions under realistic user load.

Finally, the adversarial study covered one static repository-context attack against one client/model configuration. It did not test adaptive multi-turn attacks, malicious tool-return text, poisoned job logs, compromised MCP descriptions, or attacks against production credentials. Broader adversarial evaluation remains necessary. Other high-value generalization experiments are a second model/client replication of Phases 1 and 3 and a second workload class such as GPU training or MPI using Slurm-cgroup-scoped PSI. Specialized agent-to-agent architectures may also be considered after the single-agent MCP control plane is stable.

9 Conclusion

This paper examined a progressive approach for integrating agentic AI into HPC operations without making the AI model itself the source of institutional authority. A five-phase case study separated evidence selection, evidence interpretation, authorization-sensitive behavior, technical enforcement, and response to adversarial repository context.

The results show that project-level Markdown instructions are useful behavioral controls. Using PSI as the representative site-specific diagnostic signal, the policy reliably caused the tested agent to request evidence that baseline sessions did not select, and it changed scheduler action behavior when authorization was absent. The results also show the limit of that mechanism. When identical measured evidence was supplied, both governed and baseline sessions interpreted it correctly, and when an explicit action challenge forced all sessions to attempt the same scheduler operation, Markdown did not become the enforcement layer.

The MCP service provided that enforcement boundary. Every unauthorized scheduler request in the formal enforcement challenge was denied, every authorized request was allowed exactly once, and no direct scheduler bypass occurred within the controlled experiment. In the adversarial extension, malicious repository instructions increased authorization-absent MCP submission attempts from 3/10 to 8/10, but all 11 unauthorized requests were denied and no direct scheduler action was observed. The external authorization result therefore remained stable even when untrusted context changed model behavior.

For research computing organizations, the practical conclusion is straightforward. Existing schedulers, identity systems, filesystem permissions, and institutional policies should remain authoritative. Project instructions can guide the model toward site practices and reduce undesirable action attempts. Measured scheduler and performance evidence can ground resource recommendations. Consequential actions should be exposed through narrow, auditable, independently authorized capabilities.

The long-term objective is therefore not to place an autonomous AI scheduler on an HPC cluster. It is to provide an AI-enabled research computing facilitation layer that can translate researcher intent into efficient, understandable, evidence-based, and policy-compliant use of existing HPC services.

Acknowledgements

Generative AI tools were used during manuscript preparation to assist with literature-search organization, reference compilation, and editorial restructuring. All cited sources, numerical results, experiment artifacts, and manuscript claims were checked by the author against primary records or retained study outputs. No AI system is listed as an author.

Statements and Declarations

Ethical considerations

This study evaluated AI-agent behavior, software tooling, and scheduler-control logic. It did not involve human participants, human data, or human tissue; institutional ethics approval was therefore not required.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Declaration of conflicting interest

The author declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding statement

The author received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors for this research.

Data availability

The public project repository contains the canonical policy, experiment harnesses, scoring materials, MCP prototype source, and formal result summaries used in this study. It is available at <https://github.com/sidpbury/hpc-ai-governance>. Large raw model-session bundles were retained during the study but are not all publicly released; a de-identified artifact release would further improve independent replication.

References

- Cappello F, Madireddy S, Underwood R, et al. (2026) EAIRA: Establishing a methodology for evaluating LLMs as scientific research assistants. *The International Journal of High Performance Computing Applications*. Epub ahead of print 8 September 2026. DOI: 10.1177/10943420261467786.
- Ehrett C, Feltus A, Dawson D, et al. (2026) BioCodex: HPC-Native Agentic Genomics Workflows via MCP RunSpecs and Asynchronous Slurm Execution. In: *Proceedings of the Practice and Experience in Advanced Research Computing 2026 (PEARC '26)*, Article 57, pp. 1–4. DOI: 10.1145/3785462.3815873.
- Gloaguen T, Mündler N, Müller M, et al. (2026) Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv preprint arXiv:2602.11988. Available at: <https://arxiv.org/abs/2602.11988> (accessed 14 September 2026).
- Heo T (2015) Control Group v2. Linux kernel documentation. Available at: <https://docs.kernel.org/admin-guide/cgroup-v2.html> (accessed 14 September 2026).
- Kamatar A, Pauloski JG, Babuji Y, et al. (2026) Empowering Scientific Workflows with Federated Agents. In: *2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 1403–1418. DOI: 10.1109/IPDPS65963.2026.00114.

- Lulla JL, Mohsenimofidi S, Galster M, et al. (2026) On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv preprint arXiv:2601.20404. Available at: <https://arxiv.org/abs/2601.20404> (accessed 14 September 2026).
- Ma H, Brace A, Siebenschuh C, et al. (2025) Connecting Large Language Model Agent to High Performance Computing Resource. arXiv preprint arXiv:2502.12280. Available at: <https://arxiv.org/abs/2502.12280> (accessed 14 September 2026).
- Model Context Protocol (2026) Model Context Protocol Specification, revision 2026-07-28. Available at: <https://modelcontextprotocol.io/specification/2026-07-28> (accessed 14 September 2026).
- Pan H, Chard R, Mello R, et al. (2025) Experiences with Model Context Protocol Servers for Science and High Performance Computing. arXiv preprint arXiv:2508.18489. DOI: 10.48550/arXiv.2508.18489.
- Pochelu P, Cartiaux H and Schleich J (2026) What artificial intelligence can do for high-performance computing systems? *Engineering Applications of Artificial Intelligence* 164(B): 113248. DOI: 10.1016/j.engappai.2025.113248.
- SchedMD (2026a) Control Group in Slurm. Slurm Workload Manager documentation. Available at: <https://slurm.schedmd.com/cgroups.html> (accessed 14 September 2026).
- SchedMD (2026b) Control Group v2 plugin. Slurm Workload Manager documentation. Available at: https://slurm.schedmd.com/cgroup_v2.html (accessed 14 September 2026).
- Souza R, Gueroudji A, DeWitt S, et al. (2025) PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows. In: *2025 IEEE International Conference on eScience (eScience)*, pp. 467–473. DOI: 10.1109/eScience65000.2025.00093.
- Weiner J (2018) PSI – Pressure Stall Information. Linux kernel documentation. Available at: <https://docs.kernel.org/accounting/psi.html> (accessed 14 September 2026).
- Zheng Y, Hu Y, Zhang W and Quinn A (2025) Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. arXiv preprint arXiv:2509.01245. Available at: <https://arxiv.org/abs/2509.01245> (accessed 14 September 2026).
- Zheng M, Linghu F, Li S, et al. (2026) A Comparative Study on LLM-enabled HPC User Support. In: *Proceedings of the Practice and Experience in Advanced Research Computing 2026 (PEARC '26)*, Article 9, pp. 1–8. DOI: 10.1145/3785462.3815796.